



UNIT DevLab desde cero

Versión 0.0.1

Cesar Bautista

08 de abril de 2025

Contenido

1. Cocket Nova CH552	3
1.1. ¿Por qué usar el compilador SDCC?	3
1.2. Entendiendo los Lenguajes de Programación en Sistemas Embebidos	4
1.3. Requisitos	4
1.4. PinOut	5
2. Conector JST SH 1.0mm 4 pines	7
2.1. Ejemplo de conexión con la placa DualMCU ONE	8
2.2. Ejemplo de integración del conector JST SH 1.0mm 4 pines	8
3. Salidas digitales	11
3.1. Parpadeo (blink)	11
4. Modulación por ancho de pulso (PWM)	15
4.1. Implementación	15
5. Entradas digitales	19
5.1. Pull-up y Pull-down	19
5.2. Lectura de Entradas	19
5.3. Arduino IDE y SDCC	20
6. Conversión de Analógico a Digital	21
6.1. Definición de ADC	21
6.2. Arduino IDE y SDCC	22
7. I2C (Inter-Integrated Circuit)	25
7.1. Descripción General del I2C	25
7.2. Ejemplo de aplicación Pantalla SSD1306	26
7.3. Cocket Nova implementación de I2C	26
8. Ejemplo de Integración	29
9. SPI (Interfaz Periférica Serial)	33
9.1. Visión General del protocolo SPI	33
10. Interfaz USB	35
10.1. HID (Dispositivo de Interfaz Humano)	35
10.2. CDC (Clase de Dispositivo de Comunicación)	36

11. Control WS2812	37
11.1. Control WS2812 con SDCC	38
12. Cómo Generar un Informe de Error	41
12.1. Pasos para Crear un Informe de Error:	41
13. Índices y tablas	43

Bienvenido al repositorio de prácticas para la tarjeta de desarrollo CH552G. Este recurso está diseñado para proporcionar un aprendizaje completo y estructurado a aquellos interesados en aprovechar las capacidades de los microcontroladores de bajos recursos, con un enfoque en una amplia gama de aplicaciones prácticas. Cada práctica se centra en aspectos específicos, como la programación de LEDs, el uso de sensores analógicos, aplicaciones en sistemas de monitoreo y la comunicación con pantallas, entre otros.

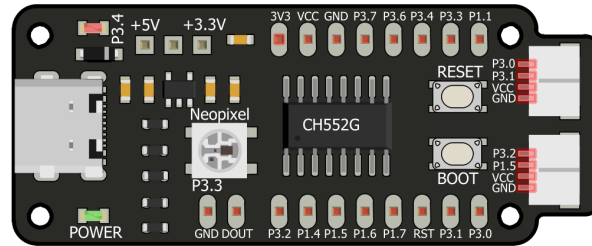


Figura 1: Cocket Nova (Imagen de referencia)

El objetivo de este repositorio es ofrecer un recurso integral que permita a los usuarios explorar y experimentar con diversas funcionalidades de la tarjeta CH552G. A través de ejemplos prácticos y explicaciones detalladas, los usuarios podrán adquirir habilidades valiosas en programación y electrónica, lo que les permitirá desarrollar proyectos innovadores y creativos.

Cocket Nova CH552

Truco: Esta guía está diseñada para personas con un conocimiento básico de programación y electrónica. Es ideal para aquellos interesados en adentrarse en sistemas embebidos y programación de microcontroladores.

Esta es una excelente guía para programadores principiantes, centrada en el uso del compilador SDCC en entornos Windows y Linux. Aquí, puedes encontrar excelentes referencias y ejemplos junto con documentación completa enfocada en desarrollar tecnología para sistemas embebidos. Este curso cubre todo, desde la instalación y configuración del compilador hasta la gestión de dependencias del proyecto y el desarrollo de código. Es un recurso valioso que te guiará a través del proceso de desarrollo utilizando tecnología de alta calidad, asegurando proyectos duraderos y robustos.

1.1 ¿Por qué usar el compilador SDCC?

El Small Device C Compiler (SDCC) es una herramienta muy reconocida en el campo del desarrollo de sistemas embebidos. Aquí hay varias razones por las que podrías elegir usar el compilador SDCC para tus proyectos:

1. **Gratis y de Código Abierto:** SDCC está disponible de forma gratuita y es de código abierto, lo que significa que puedes usarlo sin costos de licencia y contribuir a su desarrollo si lo deseas.
2. **Amplio Soporte para Microcontroladores:** Soporta una amplia gama de microcontroladores, incluidos los populares como el CH552, lo que lo convierte en una opción versátil para diversos proyectos.
3. **Facilidad de Uso:** SDCC es conocido por su interfaz amigable y configuración sencilla, lo que ayuda a los desarrolladores a comenzar rápidamente.
4. **Comunidad Activa y Documentación:** Con una comunidad activa y documentación extensa, puedes encontrar soporte y recursos para ayudarte a resolver cualquier problema que encuentres.
5. **Compatibilidad:** SDCC es compatible con muchas otras herramientas y entornos, permitiendo una integración fluida en flujos de trabajo existentes.

1.2 Entendiendo los Lenguajes de Programación en Sistemas Embebidos

Para desarrollar sistemas embebidos efectivos, es crucial entender los diferentes tipos de lenguajes de programación utilizados:

1.2.1 Lenguaje Máquina

El lenguaje máquina, también conocido como código máquina, es el nivel más fundamental de programación. Las instrucciones se escriben en patrones binarios, que son combinaciones de 1s y 0s. Estos patrones corresponden a niveles de voltaje ALTO y BAJO que el microcontrolador o microprocesador puede interpretar directamente. Este lenguaje es el más difícil de usar para los humanos debido a su complejidad y falta de legibilidad.

1.2.2 Lenguaje Ensamblador

El lenguaje ensamblador es un paso por encima del lenguaje máquina, proporcionando un formato más legible para los humanos. Utiliza **mnemónicos** y códigos hexadecimales para representar instrucciones del lenguaje máquina. Por ejemplo, el lenguaje ensamblador del microcontrolador 8051 incluye una combinación de palabras similares al inglés llamadas mnemónicos y números hexadecimales. A pesar de ser más legible que el lenguaje máquina, aún requiere un conocimiento profundo de la arquitectura del microcontrolador.

1.2.3 Lenguaje de Alto Nivel

Los lenguajes de alto nivel simplifican la programación al abstraer los detalles intrincados de la arquitectura del microcontrolador. Estos lenguajes utilizan palabras y declaraciones familiares, lo que los hace más fáciles de aprender y usar. Ejemplos de lenguajes de alto nivel incluyen BASIC, C, Pascal, C++ y Java. Los programas escritos en lenguajes de alto nivel se traducen a código máquina mediante un compilador, cerrando la brecha entre el código amigable para los humanos y las instrucciones comprensibles para la máquina.

Al comprender estos **diferentes niveles de lenguajes de programación**, puedes elegir el más adecuado para tu proyecto, equilibrando la facilidad de uso con el nivel de control que necesitas sobre el hardware.

1.3 Requisitos

- **Python** (Instalación de Paquetes y Entornos)]
- Instalación de Controladores para el Compilador SDCC
- Docker (Instalación de Paquetes y Entornos)
- Utilización de Controladores del Sistema Operativo
- Comprensión de Electrónica Básica

1.4 PinOut

La placa de desarrollo del microcontrolador CH552G tiene un total de 16 pines, cada uno con una función específica. A continuación, el diagrama de pines y su respectiva función:

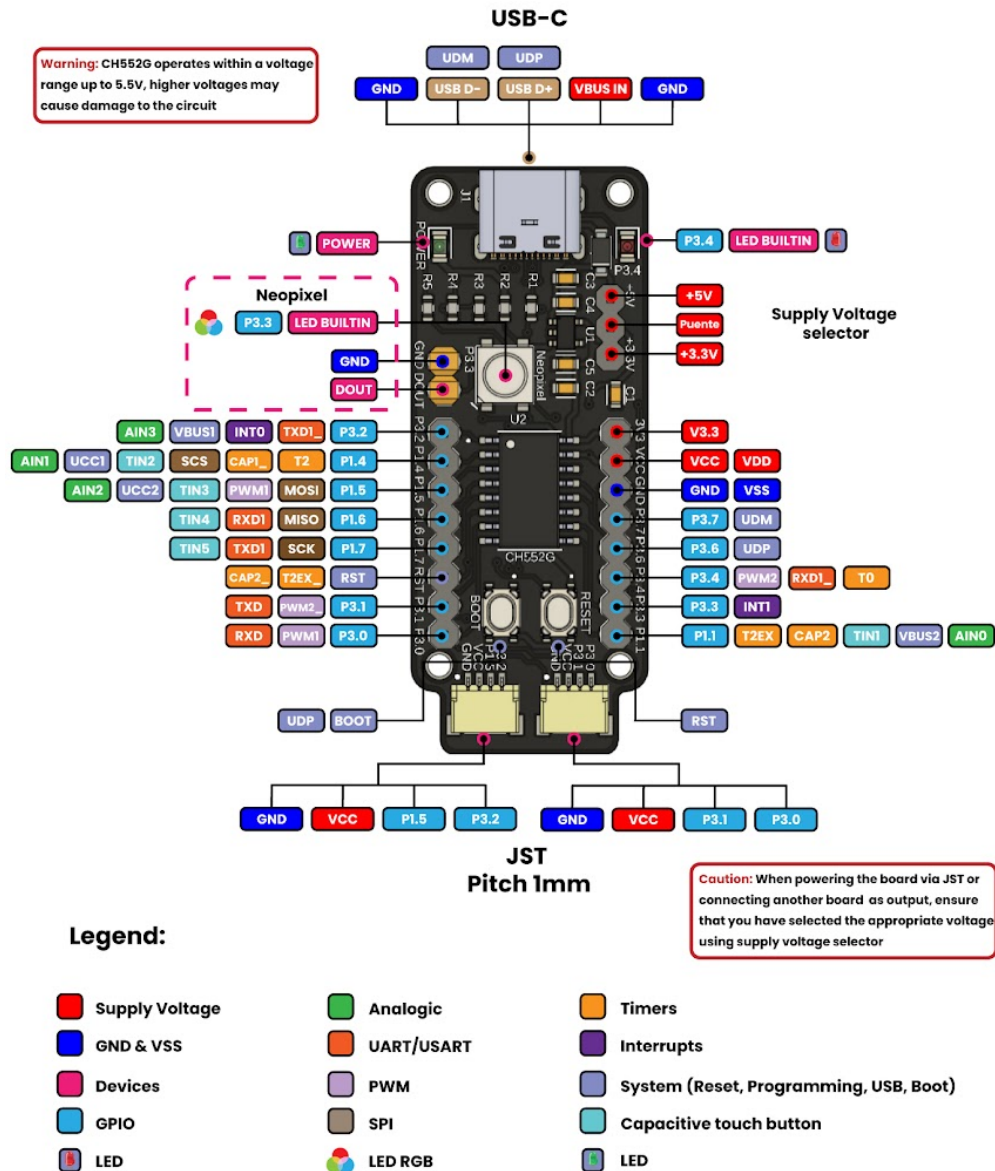


Figura 1.1: Cocket Nova CH552 PinOut

Conector JST SH 1.0mm 4 pines

El conector JST SH 1.0mm 4 pines es un conector que se utiliza para conectar las placas **Cocket Nova** a dispositivos como sensores, actuadores y otros periféricos. El conector tiene un paso de 1.0mm y se usa comúnmente en dispositivos electrónicos pequeños.

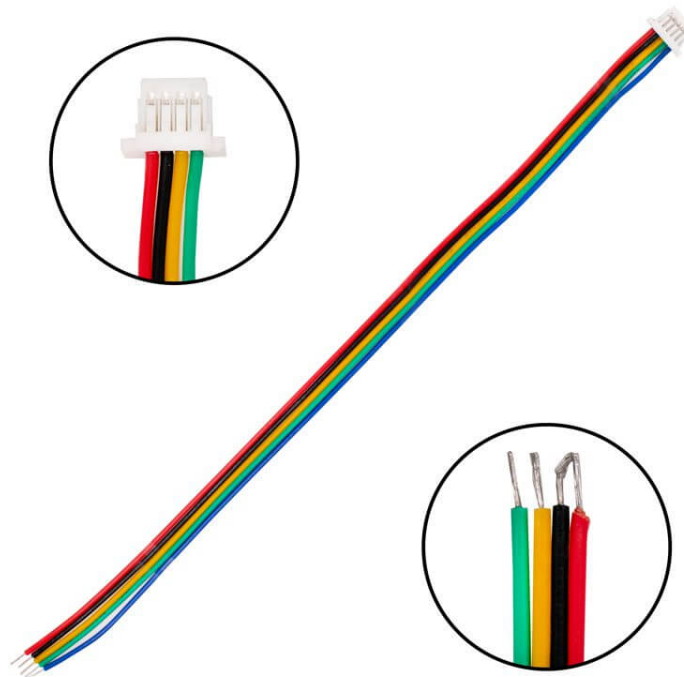


Figura 2.1: JST SH 1.0mm 4 pines

2.1 Ejemplo de conexión con la placa DualMCU ONE

Para conectar el conector JST SH 1.0mm 4 pines a la placa DualMCU ONE, siga los pasos a continuación:

1. **Identificar el conector:** El conector JST SH 1.0mm 4 pines tiene un paso de 1.0mm y 4 pines.

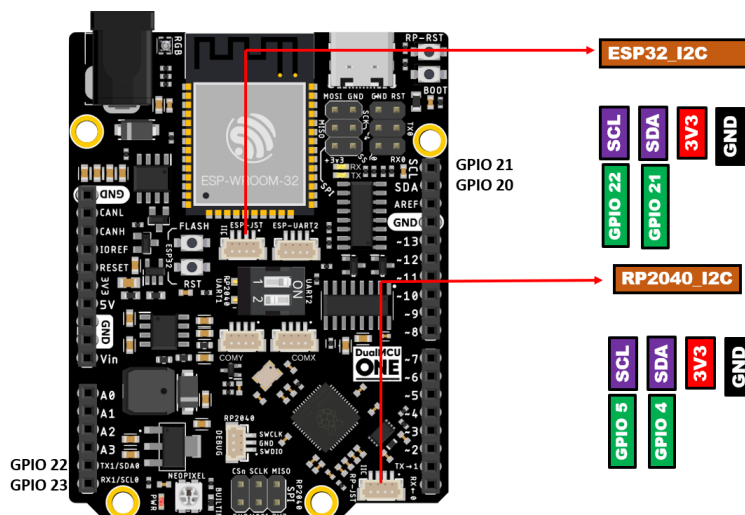


Figura 2.2: JST SH 1.0mm 4 pines (ejemplo para comunicación I2C)

2. **Conectar el conector:** Alinee el conector con los pines en la placa DualMCU ONE y empuje suavemente el conector sobre los pines.

Advertencia: A veces los colores del conector no corresponden a los pines. Tenga cuidado al conectar el conector a la placa.

3. **Verificar la conexión:** Después de conectar el conector JST SH 1.0mm 4 pines a la placa DualMCU ONE, verifique la conexión comprobando los pines en la placa.

Nota: El conector JST SH 1.0mm 4 pines es compatible con la placa desarrollada Cocket Nova.

2.2 Ejemplo de integración del conector JST SH 1.0mm 4 pines

Este ejemplo demuestra cómo integrar el Cargador Boost LiPo UNIT y el Monitor I2C con un ESP32 y mostrar el estado de la batería en una pantalla OLED.

Para más detalles del ejemplo, visite el siguiente enlace: [Ejemplo de integración](#)

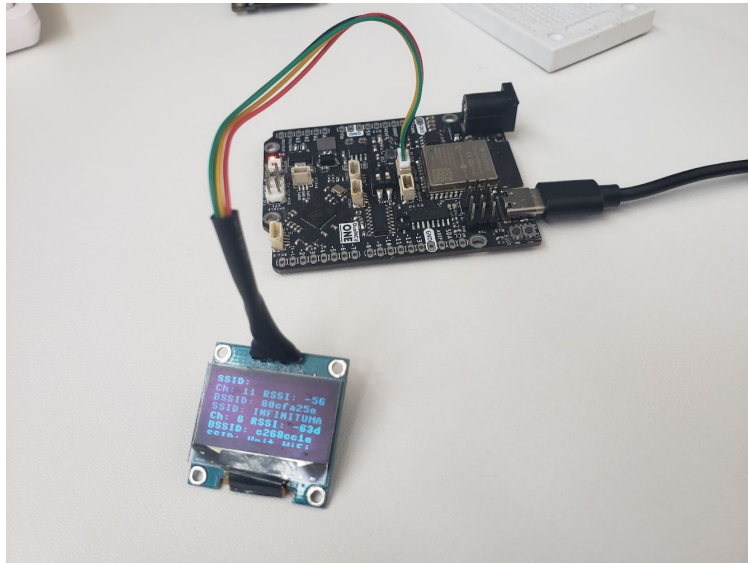


Figura 2.3: JST SH 1.0mm 4 pines conectado

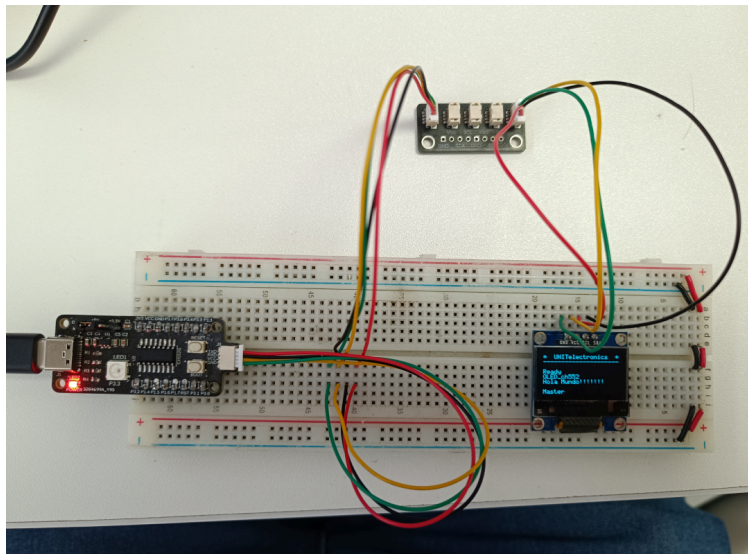


Figura 2.4: OLED a Cocket Nova

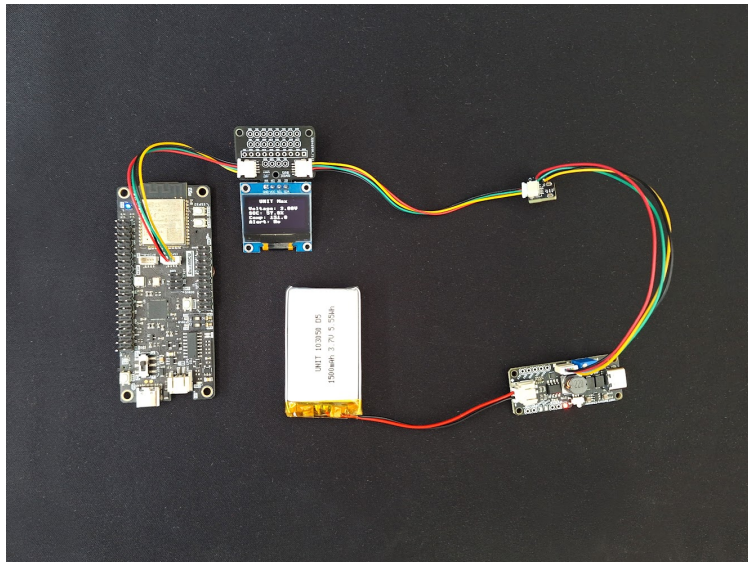


Figura 2.5: Ejemplo de integración deL conector JST a la placa DualMCU

Salidas digitales

Las salidas digitales son una forma de interactuar con el mundo exterior. En la mayoría de los microcontroladores, las salidas digitales se utilizan para encender o apagar LEDs, activar relés, controlar motores y más.

Truco: Open-drain y Open-collector

Algunos microcontroladores tienen salidas de drenaje abierto (open-drain) o colector abierto (open-collector). Estas salidas son útiles para la conexión de dispositivos de alta corriente o para la comunicación bidireccional.

- **Open-drain:** La salida puede conectarse a tierra (GND) pero no a VCC.
 - **Open-collector:** La salida puede conectarse a VCC pero no a tierra (GND).
-

Advertencia: MicroPython no se encuentra disponible para la placa de desarrollo Cocket Nova su ejemplo es solo para SDCC.

3.1 Parpadeo (blink)

Un parpadeo de LED es un proyecto común para comenzar con microcontroladores. Lo que no te dicen es la equivalencia de un parpadeo en diferentes plataformas. Para Arduino IDE o SDCC puede diferir en la cantidad de líneas de código, pero el resultado es el mismo: un LED que parpadea.

Truco: Con frecuencia, las tarjetas de desarrollo tienen un LED integrado en un pin específico, como el pin 13 en Arduino Uno o el pin 25 en la tarjeta de desarrollo Raspberry Pi Pico.

En la tarjeta de desarrollo Cocket Nova, el LED integrado está conectado al pin 34. Para encenderlo, se debe configurar el pin como salida y luego alternar su estado entre alto (HIGH) y bajo (LOW).

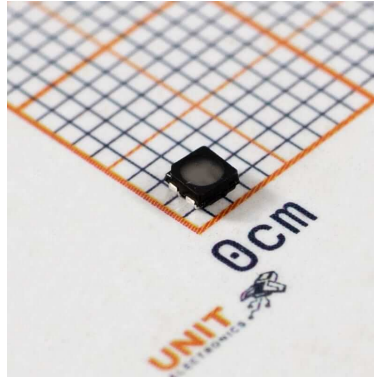


Figura 3.1: RGB_LED

3.1.1 Arduino IDE y SDCC

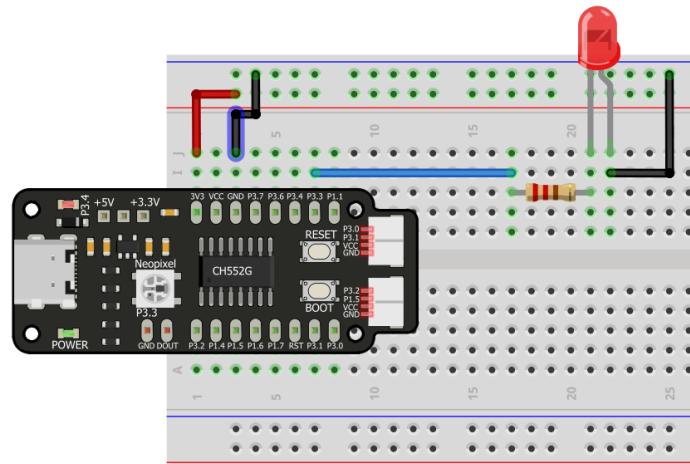


Figura 3.2: LEDs

C++

```
#define LED_BUILTIN 34

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(500);
  digitalWrite(LED_BUILTIN, LOW);
  delay(500);
}
```

SDCC


```
#include "src/system.h"
#include "src/gpio.h"
#include "src/delay.h"

#define PIN_LED P34

void main(void)
{
    CLK_config();
    DLY_ms(5);

    PIN_output(PIN_LED);
    while (1)
    {
        PIN_toggle(PIN_LED);
        DLY_ms(500);
    }
}
```

3.1.2 Aplicaciones

Contador binarios 4 bits

Modulación por ancho de pulso (PWM)

La modulación por ancho de pulso (PWM) es una técnica utilizada para controlar la cantidad de energía entregada a un dispositivo. En los microcontroladores, el PWM se utiliza para controlar la velocidad de los motores, el brillo de los LEDs y más.

Advertencia: El soporte de ubicación para salidas PWM depende de la placa de desarrollo. Revisar la documentación de la placa para conocer los pines PWM disponibles.

4.1 Implementación

4.1.1 Arduino IDE y SDCC

SDCC

```
#include <stdio.h>
#include "src/config.h"
#include "src/system.h"
#include "src/gpio.h"
#include "src/delay.h"
#include "src/pwm.h"

#define MIN_COUNTER 10
#define MAX_COUNTER 254
#define STEP_SIZE 10

void change_pwm(int hex_value)
{
    PWM_write(PIN_PWM, hex_value);
}
```

(continúe en la próxima página)

(proviene de la página anterior)

```
void main(void)
{
    CLK_config();
    DLY_ms(5);
    PWM_set_freq(1);
    PIN_output(PIN_PWM);
    PWM_start(PIN_PWM);
    PWM_write(PIN_PWM, 0);
    while (1)
    {
        for (int i = MIN_COUNTER; i < MAX_COUNTER; i+=STEP_SIZE)
        {
            change_pwm(i);
            DLY_ms(20);
        }
        for (int i = MAX_COUNTER; i > MIN_COUNTER; i-=STEP_SIZE)
        {
            change_pwm(i);
            DLY_ms(20);
        }
    }
}
```

C++

```
#define led 34

int brightness = 0;
int fadeAmount = 5;

void setup() {
    pinMode(led, OUTPUT);
}

void loop() {
    analogWrite(led, brightness);
    brightness = brightness + fadeAmount;
    if (brightness <= 0 || brightness >= 255) {
        fadeAmount = -fadeAmount;
    }
    delay(30);
}
```

4.1.2 Aplicaciones

Controlador de velocidad de motor

Entradas digitales

Las entradas digitales son una forma de interactuar con el mundo exterior. En la mayoría de los microcontroladores, las entradas digitales se utilizan para leer el estado de un botón, un sensor digital, un interruptor y más.

5.1 Pull-up y Pull-down

La mayoría de los microcontroladores tienen resistencias pull-up y pull-down internas. Estas resistencias se utilizan para mantener un valor lógico alto o bajo en una entrada digital cuando no se aplica una señal externa.

- **Pull-up:** La resistencia pull-up conecta la entrada a VCC (nivel alto).
- **Pull-down:** La resistencia pull-down conecta la entrada a tierra (nivel bajo).

Estas resistencias pueden activarse o desactivarse mediante software. En algunos microcontroladores, como el CH552, las resistencias pull-up y pull-down se pueden configurar con un valor de resistencia específico.

También es posible utilizar resistencias externas para pull-up y pull-down. Esto es útil cuando se necesita un valor de resistencia específico o cuando las resistencias internas no son suficientes.

5.2 Lectura de Entradas

La lectura de una entrada digital en un microcontrolador es un proceso sencillo. La entrada puede estar en uno de dos estados: alto (HIGH) o bajo (LOW). En la mayoría de los microcontroladores, el estado alto corresponde a un valor lógico de 1 y el estado bajo a un valor lógico de 0.

5.3 Arduino IDE y SDCC

C++

```
#include <Serial.h>

void setup() {
  // No need to init USBSerial
  pinMode(33, INPUT);
  pinMode(34, OUTPUT);
}

void loop() {
  // Leer el valor del botón en una variable
  int sensorVal = digitalRead(11);
  // Imprimir el valor del botón en el monitor serial
  USBSerial.println(sensorVal);
  if (sensorVal == HIGH) {
    digitalWrite(33, LOW);
  } else {
    digitalWrite(33, HIGH);
  }

  delay(10);
}
```

SDCC

```
#include "src/system.h"
#include "src/gpio.h"
#include "src/delay.h"

#define PIN_LED P34
#define PIN_BUTTON P33

void main(void)
{
  CLK_config();
  DLY_ms(5);
  PIN_input(PIN_BUTTON);
  PIN_output(PIN_LED);
  while (1)
  {
    if (PIN_read(PIN_BUTTON)){
      PIN_high(PIN_LED);
    }
    else{
      PIN_low(PIN_LED);
    }
  }
}
```

Conversión de Analógico a Digital

6.1 Definición de ADC

La conversión de analógico a digital (ADC) es un proceso que convierte señales analógicas en valores digitales. Los microcontroladores utilizan ADC para leer señales analógicas de sensores y otros dispositivos.

Advertencia: El voltaje de referencia del ADC varía según el microcontrolador. Consulta la hoja de datos del microcontrolador para obtener información específica.

6.1.1 Cuantificación y Codificación de Señales Analógicas

Las señales analógicas son continuas y pueden tomar cualquier valor dentro de un rango dado. En cambio, las señales digitales son discretas y solo pueden adoptar valores específicos. La conversión de una señal analógica a digital implica dos pasos: cuantificación y codificación.

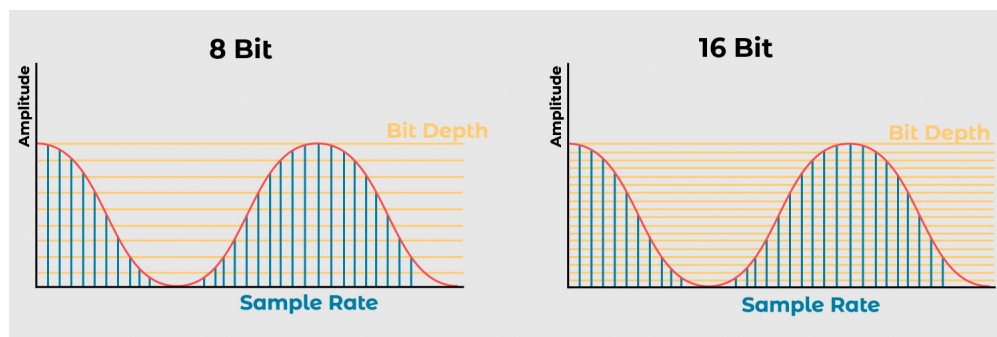


Figura 6.1: Conversión de Analógico a Digital

Tabla 6.1: Ejemplos de Codificación y Cuantificación

Resolución	Niveles de Cuantificación	Código Digital
8 bits	256	0x00 a 0xFF
12 bits	4,096	0x000 a 0xFFF
16 bits	65,536	0x0000 a 0xFFFF

Cuantificación

Divide la señal analógica en niveles discretos. El número de niveles determina la resolución del ADC.

Nota: La resolución de un ADC se mide en bits y se calcula como 2^n , donde n es el número de bits.

Tabla 6.2: Resoluciones de ADC

Resolución	Niveles de Cuantificación	Descripción
8 bits	256	Un ADC de 8 bits tiene 256 niveles de cuantificación, lo que significa que puede representar la señal analógica con 256 valores diferentes.
12 bits	4,096	Un ADC de 12 bits tiene 4,096 niveles de cuantificación, lo que permite representar la señal analógica con 4,096 valores distintos.
16 bits	65,536	Un ADC de 16 bits tiene 65,536 niveles de cuantificación, permitiendo representar la señal analógica con 65,536 valores distintos.

Codificación

Asigna un código digital a cada nivel de cuantificación. Este código digital representa el valor de la señal analógica en dicho nivel.

6.2 Arduino IDE y SDCC

El ADC del microcontrolador CH552 tiene una resolución de 8 bits, lo que significa que puede representar valores entre 0 y 255. Para leer un valor analógico, se utiliza la función *analogRead()* en Arduino IDE o *ADC_read()* en SDCC.

C++

```
#define LED_BUILTIN 34

int sensorPin = 11;
int ledPin = LED_BUILTIN;
int sensorValue = 0;

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(sensorPin, INPUT);
}
```

(continúe en la próxima página)

(proviene de la página anterior)

```
}  
  
void loop() {  
    sensorValue = analogRead(sensorPin);  
    digitalWrite(ledPin, HIGH);  
    delay(sensorValue);  
    digitalWrite(ledPin, LOW);  
    delay(sensorValue);  
}
```

SDCC

```
#include "src/system.h"  
#include "src/gpio.h"  
#include "src/delay.h"  
  
#define PIN_ADC P11  
  
void main(void)  
{  
    CLK_config();  
    DLY_ms(5);  
  
    ADC_input(PIN_ADC);  
    ADC_enable();  
  
    while (1)  
    {  
        int data = ADC_read(); // Leer valor ADC (0 - 255, 8 bits)  
    }  
}
```

I2C (Inter-Integrated Circuit)

7.1 Descripción General del I2C

I2C (Inter-Integrated Circuit o Inter-IC) es un bus de comunicación serial síncrono, multi-maestro, multi-esclavo, de conmutación de paquetes y referencia única. Se utiliza comúnmente para conectar periféricos de baja velocidad a procesadores y microcontroladores. La gran mayoría de las placas de desarrollo de UNIT ELECTRONICS, como DualMCU, DualMCU ONE y Cocket Nova, ofrecen capacidades de comunicación I2C, lo que te permite interconectar una amplia gama de dispositivos I2C. A pesar de que la Cocket Nova no cuenta con I2C nativo, se implementa bajo una técnica llamada bit-banging.

Bit-banging es una técnica de programación en la que se manipulan directamente los pines de un microcontrolador para implementar un protocolo de comunicación. En el caso de I2C, se utilizan dos pines, SDA (Serial Data) y SCL (Serial Clock), para la comunicación entre dispositivos.

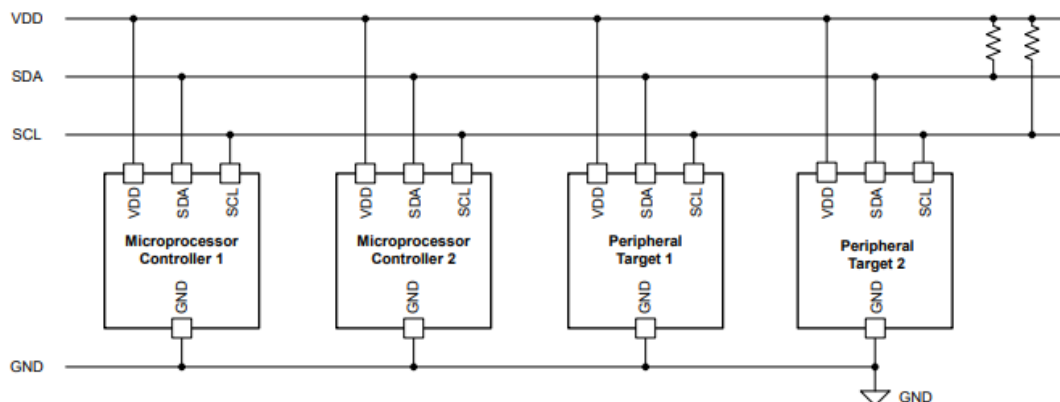


Figura 7.1: Pines I2C Imagen obtenida de [Application Note A Basic Guide to I2C](#)

Nota: Las tarjetas de desarrollo de UNIT ELECTRONICS son compatibles con [Conector JST](#)

7.2 Ejemplo de aplicación Pantalla SSD1306



Figura 7.2: Pantalla SSD1306

La pantalla OLED monocromática de 128x64 píxeles equipada con un controlador SSD1306 se conecta mediante un conector JST de 1.25mm de 4 pines. La siguiente tabla proporciona los detalles de conexión para la pantalla.

Tabla 7.1: Asignación de Pines de la Pantalla SSD1306

Pin	Conexión
1	GND
2	VCC
3	SDA
4	SCL

7.3 Cocket Nova implementación de I2C

```
#include "src/config.h"
#include "src/system.h"
#include "src/gpio.h"
#include "src/delay.h"
#include "src/oled.h"

void main(void) {
    CLK_config();
    DLY_ms(5);
}
```

(continúe en la próxima página)

(proviene de la página anterior)

```

OLED_init();

OLED_print("*  UNITElectronics  *");
OLED_print("-----\n");
OLED_print("Ready\n");
while(1) {

}
}

```

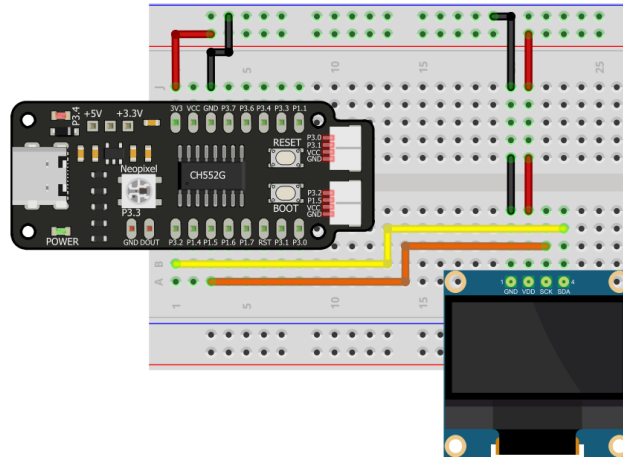


Figura 7.3: Pantalla OLED Cocket Nova

Ejemplo de Integración

Este ejemplo demuestra cómo integrar el Cargador Boost LiPo & Monitor I2C de UNIT con un ESP32 y mostrar el estado de la batería en una pantalla OLED.

Prudencia: Dado que el estándar Qwiic opera a 3.3V, se recomienda usar un regulador de voltaje después de la salida JST del módulo para reducir VSYS a 3.3V y proteger los dispositivos Qwiic conectados.

Pasos para integrar el **Cargador Boost LiPo & Monitor I2C** de UNIT con el ESP32 y la pantalla OLED usando el conector JST y la comunicación I2C:

1. **Conectar el Conector JST:** Conecta el conector JST a la placa DualMCU usando el **conector JST SH de 1.0mm y 4 pines**.
2. **Conectar el ESP32:** Conecta el ESP32 a la placa DualMCU mediante el protocolo de comunicación I2C.

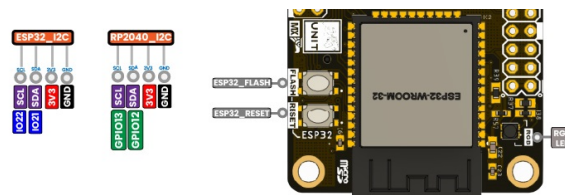


Figura 8.1: Conector JST SH de 1.0mm y 4 pines conectado al ESP32.

3. **Soldar la Configuración de Pines de la Batería:** Suelda las configuración apropiada para el cargador de batería. Configura la corriente de carga (0.2 mA) según las especificaciones de la batería.

Advertencia: Asegúrate de verificar la polaridad de la batería antes de conectarla al cargador.

4. **Conectar la Pantalla OLED:** Conecta la pantalla OLED al ESP32 usando el protocolo de comunicación I2C.
5. **Subir el Código al ESP32:** Copia el siguiente código al ESP32 y ejecútalo usando Thonny:

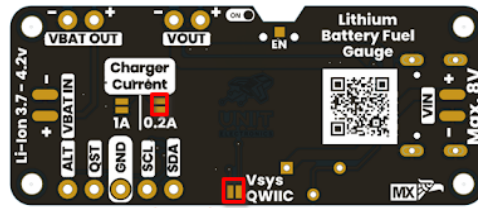


Figura 8.2: Configuración de Pines de la Batería

```
import machine
import time
from ocs import SSD1306_I2C
from max1704x import max1704x

i2c = machine.SoftI2C(sda=machine.Pin(21), scl=machine.Pin(22))
oled = SSD1306_I2C(128, 64, i2c)

my_sensor = max1704x(sda_pin=21, scl_pin=22)

def update_display():
    oled.fill(0)
    oled.text('UNIT Max', 25, 0)

    vcell = my_sensor.getVCell()
    soc = my_sensor.getSoc()
    compensate_value = my_sensor.getCompensateValue()
    alert_threshold = my_sensor.getAlertThreshold()
    in_alert = my_sensor.inAlert()

    oled.text("Voltaje: {:.2f}V".format(vcell), 0, 16)
    oled.text("SOC: {:.1f}%".format(soc), 0, 26)
    oled.text("Comp: {:.1f}".format(compensate_value), 0, 36)
    oled.text("Alerta: {}".format("Si" if in_alert else "No"), 0, 46)

    oled.show()

while True:
    update_display()
    time.sleep(2)
```

6. **Monitorear el Estado de la Batería:** La pantalla OLED mostrará el estado de la batería, incluyendo el voltaje, estado de carga, valor de compensación y estado de alerta.
7. **Cargar la Batería:** Conecta la batería al cargador y monitorea el estado de carga usando la pantalla OLED.

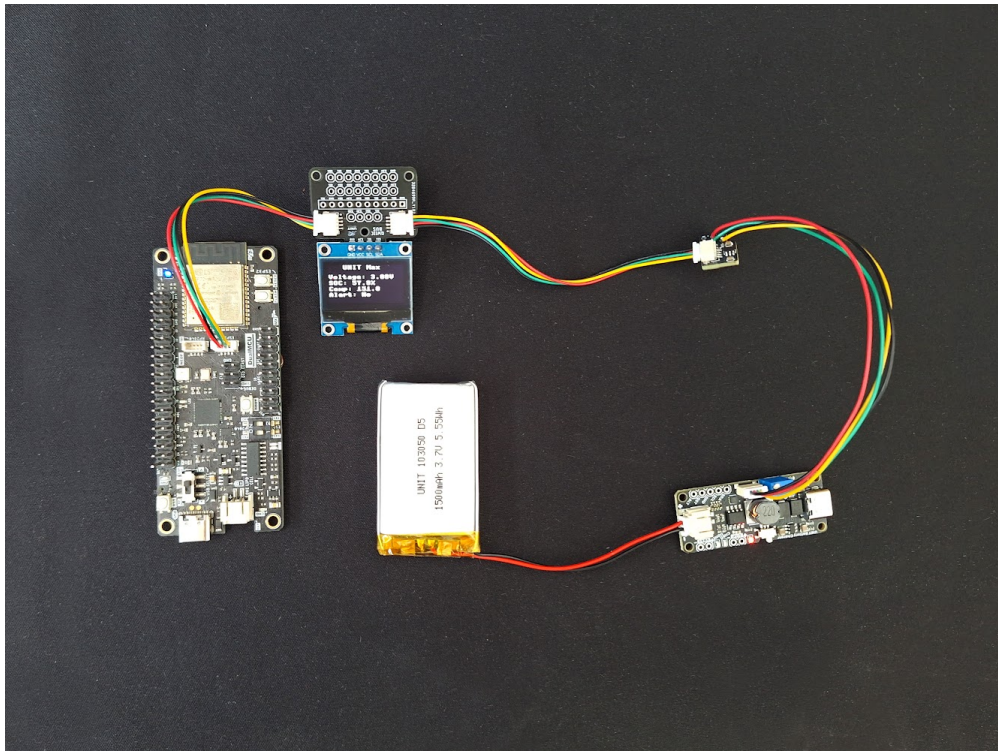


Figura 8.3: Ejemplo de Integración del Cargador Boost LiPo & Monitor I2C de UNIT con ESP32 y Pantalla OLED.

SPI (Interfaz Periférica Serial)

Advertencia: El soporte para la tarjeta de desarrollo Cocket Nova (CH552) se encuentra en desarrollo. Por favor, mantente atento a futuras actualizaciones.

9.1 Visión General del protocolo SPI

SPI (Interfaz Periférica Serial) es un bus de comunicación síncrono, full-duplex y maestro-esclavo. Se utiliza comúnmente para conectar microcontroladores a periféricos como sensores, pantallas y dispositivos de memoria. La placa de desarrollo DualMCU ONE ofrece capacidades de comunicación SPI, lo que te permite interconectar una amplia gama de dispositivos SPI.

La interfaz USB es un protocolo de comunicación que permite la conexión de dispositivos electrónicos a una computadora. En el caso de los microcontroladores, la interfaz USB permite la comunicación con una computadora para la transferencia de datos, actualización de firmware, depuración, etc.

10.1 HID (Dispositivo de Interfaz Humano)

HID (Human Interface Device) es un protocolo de comunicación USB que permite a los dispositivos electrónicos interactuar con un usuario. Los dispositivos HID incluyen teclados, ratones, joysticks, gamepads, etc.

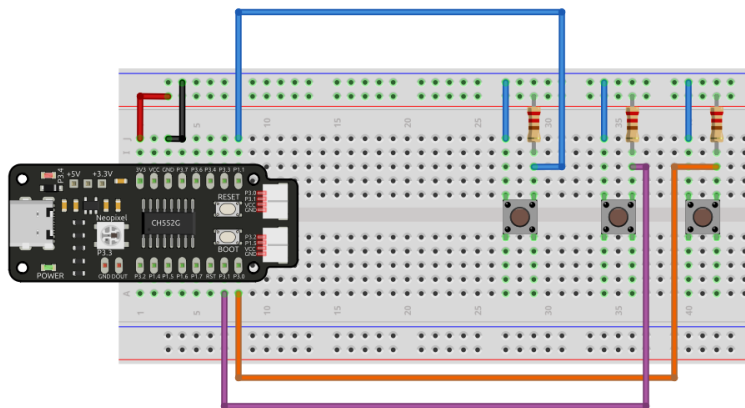


Figura 10.1: Ejemplo de Dispositivo HID

10.2 CDC (Clase de Dispositivo de Comunicación)

CDC (Communication Device Class) es una clase de dispositivos USB que permite la comunicación entre un dispositivo y una computadora. Los dispositivos CDC incluyen módems, adaptadores de red, etc.

CAPÍTULO 11

Control WS2812

El led WS2812 es un led RGB que se puede controlar con un solo pin de datos. Es muy popular en proyectos de iluminación y decoración. En este tutorial, aprenderás cómo controlar un led WS2812 con MicroPython y Arduino.

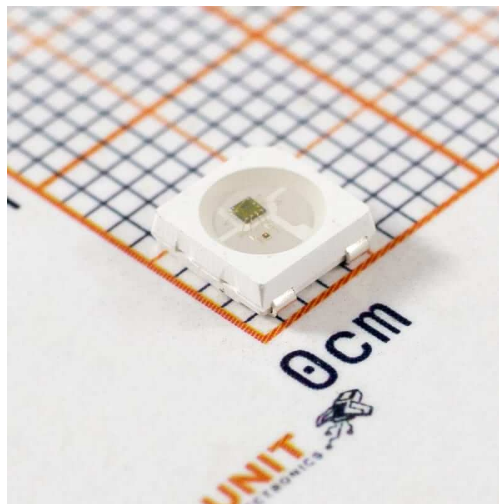


Figura 11.1: Tira LED WS2812

11.1 Control WS2812 con SDCC

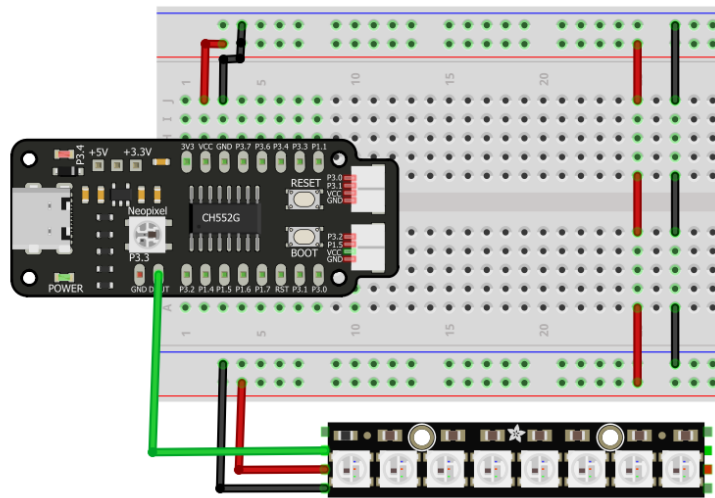


Figura 11.2: Tira LED WS2812

SDCC

```
#include "src/config.h"
#include "src/system.h"
#include "src/delay.h"
#include "src/neo.h"
#include <stdlib.h>

#define delay 100
#define NeoPixel 2
#define level 100

void randomColorSequence(void) {
    for(int j=0;j<NeoPixel;j++){
        uint8_t red = rand() % level;
        uint8_t green = rand() % level;
        uint8_t blue = rand() % level;
        uint8_t num = rand() % NeoPixel;

        for(int i=0; i<num; i++){
            NEO_writeColor(0, 0, 0);
        }
        NEO_writeColor(red, green, blue);
        DLY_ms(delay);
        NEO_writeColor(0, 0, 0);
    }

    for(int l=0; l<9; l++){
        NEO_writeColor(0, 0, 0);
    }
}
```

(continúe en la próxima página)

(proviene de la página anterior)

```
}

void main(void) {
  NEO_init();
  CLK_config();
  DLY_ms(delay);

  while (1) {
    randomColorSequence();
    DLY_ms(10);
  }
}
```

Arduino-IDE

Cómo Generar un Informe de Error

Esta guía explica cómo generar un informe de error utilizando repositorios de GitHub.

12.1 Pasos para Crear un Informe de Error:

1. **Acceder al Repositorio de GitHub** Navega al repositorio de GitHub donde se aloja el proyecto.
2. **Abrir la Pestaña de Issues** Haz clic en la pestaña «Issues» ubicada en el menú del repositorio.
3. **Crear un Nuevo Issue**
 - Haz clic en el botón «New Issue».
 - Proporciona un título claro y conciso para el issue.
 - **Añade una descripción detallada, incluyendo información relevante como:**
 - Pasos para reproducir el error.
 - Resultados esperados y reales.
 - Cualquier registro, captura de pantalla o archivo relacionado.
4. **Enviar el Issue** Una vez que el formulario esté completo, haz clic en el botón «Submit».

El equipo de desarrollo o los mantenedores revisarán el issue y tomarán las medidas apropiadas para abordarlo.

CAPÍTULO 13

Índices y tablas

- genindex
- modindex
- search